

Design Notes:

BEAVER'S CHOICE PAPER COMPANY MULTI-AGENT SYSTEM

Design, Evaluation and Reflection Report

1. SYSTEM OVERVIEW

This project implements a four-agent inventory, quoting and sales workflow using the smolagents framework and an SQLite business database. The architecture stays below the five-agent limit and separates responsibilities so that pricing cannot change stock, inventory management cannot finalize customer sales, and the sales agent cannot bypass feasibility checks.

The system uses deterministic Python rules inside smolagents tools. This design retains a clear multi-agent/tool architecture while protecting financial and inventory transactions from unpredictable free-form model output. All inputs and customer responses remain text based.

2. AGENT ARCHITECTURE AND DECISION PROCESS

Customer Inquiry Orchestrator Agent

- Receives the customer request and dataset context.
- Extracts the request date, required delivery date, quantities and product names.
- Normalizes customer wording to exact catalog names.
- Delegates first to Inventory, then Quoting, and finally Sales.
- Stops immediately when an item is unsupported or a feasibility check fails.
- Produces the final customer-safe explanation.

Inventory Agent

- Owns inventory checks, supplier lead-time assessment and reordering decisions.
- Checks projected stock at the requested deadline rather than only current stock.
- Calculates shortages, verifies supplier arrival dates and checks whether the combined replenishment purchase is affordable.
- Creates stock-order transactions only after every requested line is feasible.

Quoting Agent

- Owns customer pricing and bulk discounts.
- Searches historical quotes using the customer's job/event context.
- Calculates a consistent catalog-based subtotal and applies a transparent volume discount of 2%, 5%, 7%, 10% or 12% at defined thresholds.
- Returns the quote total, saving and history-use information to the orchestrator.

Sales Agent

- Owns final stock verification and sale completion.

- Rechecks every line at the planned fulfillment date.
- Runs a private financial health check before recording transactions.
- Records a sales transaction for every fulfilled product line.

The sequence is intentional: Inventory -> Quote -> Sale. A quote is generated only after the order is feasible, and revenue is recorded only after final verification. This avoids quoting unavailable products and prevents overselling.

3. TOOLS AND STARTER HELPER FUNCTIONS

inventory_assessment_tool

- get_all_inventory: obtains the current company-wide inventory snapshot.
- get_stock_level: calculates projected stock for each exact catalog item.
- get_supplier_delivery_date: checks whether shortages can arrive by the deadline.
- get_cash_balance: verifies that required replenishment is affordable.
- create_transaction: records approved supplier stock orders.

quote_generation_tool

- search_quote_history: retrieves comparable historical quotes by customer/event context before the pricing decision is finalized.
- The supplied paper_supplies catalog provides exact unit prices.

finalize_sale_tool

- get_stock_level: performs the final fulfillment-date stock check.
- generate_financial_report: performs a private business health check.
- create_transaction: records completed sales and reduces available inventory.

The supplied generate_sample_inventory and init_database functions remain part of database initialization. Every helper explicitly required by the rubric is called inside at least one framework tool.

4. INVENTORY, REORDERING AND FULFILLMENT RULES

The system first checks projected stock through the delivery deadline. When stock is insufficient, it calculates the exact shortage. Supplier lead time is based on the supplied helper: same day for 10 or fewer units, one day for 11-100, four days for 101-1000, and seven days for quantities above 1000.

An order is rejected when any requested item is outside the catalog, the supplier cannot meet the customer deadline, the replenishment cannot be afforded, or final stock verification fails. Rejected orders receive a reason and do not create a sales transaction. Feasible reorders are scheduled for the supplier arrival date, and the customer sale is scheduled when all lines are available.

5. QUOTING STRATEGY

The quote starts with the supplied catalog unit prices and a consistent commercial markup. Similar historical quotes are retrieved to ground the decision in prior company activity. Volume discounts encourage larger orders while remaining consistent and explainable:

- Fewer than 500 units: 2%
- 500-999 units: 5%
- 1,000-4,999 units: 7%
- 5,000-9,999 units: 10%
- 10,000 or more units: 12%

Customer responses show requested items, total price, discount and saving, whether historical quotes were available, and the promised delivery date. They never expose cash balance, internal asset totals, purchase budgets, profit margin or system errors.

6. EVALUATION RESULTS

The complete 20-row `quote_requests_sample.csv` dataset was processed in request-date order. The resulting `test_results.csv` contains one response for every request.

Observed results:

- Total requests evaluated: 20
- Successfully fulfilled orders: 9
- Unfulfilled/rejected orders: 11
- Rows showing a cash-balance change after chronological processing: at least 4
- Required minimums: 3 fulfilled, 1 unfulfilled, and 3 cash-balance changes

Examples of successful behavior include normalizing variants such as "A4 glossy paper" to "Glossy paper," applying volume discounts, reordering feasible shortages, and scheduling delivery within the stated deadline. Rejections included unsupported products such as balloons, cardboard and tickets, as well as requests whose supplier arrival date fell after the customer deadline.

The strongest aspects were safe database mutation, clear division of agent duties, full-dataset coverage, exact catalog-name normalization and transparent customer responses. The system exceeded all quantitative evaluation thresholds.

7. LIMITATIONS AND FURTHER IMPROVEMENTS

Improvement 1: Replace rule-based product normalization with a maintained synonym table plus semantic matching. The current parser intentionally rejects ambiguous products; semantic matching with a confidence threshold could suggest alternatives

without silently substituting the wrong item.

Improvement 2: Add reservations and order identifiers. Stock is currently represented through dated transactions. A separate reservations table would prevent conflicts when several accepted orders compete for the same future delivery and would allow cancellation or amendment workflows.

Improvement 3: Improve pricing intelligence. Historical quotes could be normalized by product mix, quantity, recency and customer segment, then used to recommend a bounded price range instead of only providing contextual evidence.

Improvement 4: Add proactive business advice. A reporting agent could analyze demand, stockout frequency, supplier performance and margins, then recommend revised reorder points and safety-stock levels without exposing sensitive figures to customers.

8. CONCLUSION

The four-agent design provides a modular and auditable workflow from inquiry through inventory planning, quoting and final sale. It uses all mandatory database helpers, honors supplier and inventory constraints, records financial changes, rejects impossible requests, and explains every customer-facing outcome without disclosing sensitive internal business information.